Question Bank

Contents

No.	Title	Difficulty
01	Automated Filling Line Control	Easy
02	Dual-Word Edge Detection	Easy
03	Alarm Handling	Easy
04	Jam Pump Debouncing and Runtime Maintenance	Easy
05	Weld Gap Alarm and Startup Inhibition	Easy
06	Aeration Blower Delayed Start	Easy
07	Multi-Pump Switching Control System	Medium
08	Analog Batch Processing	Medium
09	Calculate Moving Average	Medium
10	Temperature Logic Control	Medium
11	Automated Production Line Control	Medium
12	Slab Thickness Alarm and Fault History	Medium
13	Office HVAC Multi-mode Energy-Saving Control	Medium
14	IPv4 Parsing with Format and Range Checks	Medium
15	Material Proportioning Process Control	Hard
16	Flexible Processing Control	Hard
17	Recipe Management System	Hard
18	FIFO First-In-First-Out Queue	Hard
19	Mechanical Arm Control	Hard
20	Production Control	Hard

01 · Automated Filling Line Control

- **Difficulty:** Easy (easy)

Requirements

An automated filling line responsible for cleaning, filling, capping, and packaging bottles. Each step of the production line is controlled by PLC and triggered by signals from sensors and operators.
Control requirements:
When the sensor bottleSensor detects a bottle, it triggers the start of the cleaning pump and continues to work.

After cleaning is completed, the operator operates the button cleaningConfirmButton to trigger the cleaning completion signal, automatically starting the filling valve for filling.
After filling is completed, the operator operates the button fillingConfirmButton to trigger the filling completion signal, automatically starting the capping machine for capping.
After capping is completed, the operator operates the button cappingConfirmButton to trigger the capping completion signal, starting the packing machine for packaging.
After packaging is completed, the operator operates the button packingConfirmButton to trigger the packaging completion signal, lighting up the packaging completion indicator light.
After the operator picks up the packaged bottle, they press the confirmation button finishedButton to reset the indicator light and prepare for the next bottle's production cycle.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`bottleSensor`	`BOOL`	The bottle reaches the sensor, triggering a signal when a bottle is detected
Input	`cleaningConfirmButton`	`BOOL`	Operator confirmation button, used to confirm the completion of the cleaning process
Input	`fillingConfirmButton`	`BOOL`	Operator confirmation button, used to confirm the completion of the filling process
Input	`cappingConfirmButton`	`BOOL`	Operator confirmation button, used to confirm the completion of the capping process
Input	`packingConfirmButton`	`BOOL`	Operator confirmation button, used to confirm the completion of the packaging process
Input	`finishedButton`	`BOOL`	Operator confirmation button, used to confirm that the bottle has been picked up
Output	`Pump_Motor`	`BOOL`	Control output for the operation of the cleaning pump motor
Output	`Filling_Valve`	`BOOL`	Control output for the operation of the filling valve
Output	`Capping_Machine`	`BOOL`	Control output for the operation of the capping machine
Output	`Packing_Machine`	`BOOL`	Control output for the operation of the packing machine
Output	`Completion_Light`	`BOOL`	Indicator light for packaging completion

02 · Dual-Word Edge Detection

- **Difficulty:** Easy (easy)

Requirements

Write a function block FB that checks for rising and falling edges in each bit of a dual-word (DWord) input value every cycle. The function block should be able to detect and report changes, rising edges, and falling edges in the input value.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`value`	`DWORD`	The input value to be checked for changes and edge triggering
Output	`hasChanged`	`BOOL`	True indicates that the input value has changed
Output	`hasRisingEdges`	`BOOL`	True indicates that rising edges have occurred in the input value
Output	`risingBits`	`DWORD`	The bit stream of rising edges, where each bit indicates whether the corresponding input bit has experienced a rising edge (1 for a rising edge, 0 for no rising edge)
Output	`noOfRisingBits`	`USINT`	The number of rising edges in the input value
Output	`hasFallingEdges`	`BOOL`	True indicates that falling edges have occurred in the input value
Output	`fallingBits`	`DWORD`	The bit stream of falling edges, where each bit indicates whether the corresponding input bit has experienced a falling edge (1 for a falling edge, 0 for no falling edge)
Output	`noOfFallingBits`	`USINT`	The number of falling edges in the input value

03 · Alarm Handling

- **Difficulty:** Easy (easy)

Requirements

In industrial automation systems, monitoring the status of equipment is crucial. When equipment parameters exceed the normal range, the system needs to issue an alarm. To avoid frequent triggering of alarms due to minor fluctuations in parameters, an alarm management system with hysteresis function needs to be implemented.

When Value is greater than hiLevel, trigger high alarm hiAlarm.
When Value is less than hiLevel-hystWindow, exit high alarm hiAlarm.
When Value is less than loLevel, trigger low alarm loAlarm.
When Value is greater than loLevel+hystWindow, exit low alarm loAlarm.
If loLevel is greater than or equal to hiLevel, output error error=true, status=16#8001, and exit the program
If the hysteresis window value hystWindow is greater than or equal to the deviation range between hiLevel and loLevel, output error error=true, status=16#8002

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`Value`	`REAL`	Input value
Input	`hiLevel`	`REAL`	High limit alarm value
Input	`loLevel`	`REAL`	Low limit alarm value
Input	`hystWindow`	`REAL`	Hysteresis window
Output	`hiAlarm`	`BOOL`	High limit alarm
Output	`loAlarm`	`BOOL`	Low limit alarm
Output	`error`	`BOOL`	FALSE: No error occurred; TRUE: Error occurred during execution of FB
Output	`status`	`WORD`	Status code

---

04 · Jam Pump Debouncing and Runtime Maintenance

- **Difficulty:** Easy (easy)

Requirements

Control a jam transfer pump on a food production line, including viscosity-signal debouncing, emergency-stop priority, runtime accumulation and a maintenance-due indication.

1. Accept the viscosity condition only after VISCOSITY_OK has remained TRUE for DebounceTime. Invalidate the condition immediately when the input becomes FALSE.
2. Allow VFD_RUN to be TRUE only when PASTE_E_STOP_NC is TRUE and the viscosity condition is valid. An emergency stop immediately disables the run output and clears any incomplete debounce timing.
3. Accumulate PUMP_RUNTIME_ACC using the actual invocation interval CycleTime only while VFD_RUN is TRUE. Preserve the accumulated time while the pump is stopped.
4. Set PUMP_MAINT_DUE when accumulated runtime reaches 500 hours. This indication is informational and must not affect pump operation or the safety permissive.
5. At power-up, initialize the run output to OFF, accumulated runtime to zero and the maintenance indication to FALSE.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`VISCOSITY_OK`	`BOOL`	Raw signal indicating that viscosity is within the permitted range.
Input	`PASTE_E_STOP_NC`	`BOOL`	Normally closed emergency-stop circuit; TRUE permits operation and FALSE means emergency stop.
Input	`DebounceTime`	`TIME`	Required continuous valid duration for the viscosity signal; must be positive.
Input	`CycleTime`	`TIME`	Actual elapsed interval represented by this invocation; must be positive.
Output	`VFD_RUN`	`BOOL`	Pump variable-frequency-drive run command.
Output	`PUMP_MAINT_DUE`	`BOOL`	Maintenance indication when accumulated runtime reaches 500 hours.
Output	`PUMP_RUNTIME_ACC`	`LTIME`	Accumulated enabled runtime, using a long-duration time type.

---

05 · Weld Gap Alarm and Startup Inhibition

- **Difficulty:** Easy (easy)

Requirements

Control a weld-gap alarm with power-up monitoring inhibition, input debouncing during normal monitoring and emergency-stop priority.

1. Gap_Exceeded indicates that the scanner has detected a weld gap exceeding the permitted 1.5 mm value.
2. Begin with a 5-second initialization inhibition period after power-up. During this period, Operator_Alert is FALSE, Weld_Soft_Start_Active is TRUE, and gap-signal debounce time does not accumulate.
3. Resume gap monitoring after initialization. Allow the alarm only after Gap_Exceeded has remained TRUE for DebounceTime. When the signal becomes FALSE, switch off the alarm and clear debounce timing.
4. When Weld_E_Stop_NC is FALSE, the emergency stop takes priority over initialization and normal monitoring. Operator_Alert must immediately become FALSE and any incomplete debounce timing must be cleared.
5. Weld_Soft_Start_Active indicates only the power-up inhibition period; it does not represent a continuous speed ramp.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`Gap_Exceeded`	`BOOL`	Scanner indication that the gap exceeds 1.5 mm.
Input	`Weld_E_Stop_NC`	`BOOL`	TRUE means the emergency stop is inactive; FALSE means it is active.
Input	`DebounceTime`	`TIME`	Gap-signal debounce duration during normal monitoring; must be positive.
Output	`Operator_Alert`	`BOOL`	Audible and visual alarm command.
Output	`Weld_Soft_Start_Active`	`BOOL`	Monitoring-inhibition status during the 5-second power-up initialization.

---

06 · Aeration Blower Delayed Start

- **Difficulty:** Easy (easy)

Requirements

Control a wastewater-treatment aeration blower with a low-dissolved-oxygen demand, a 5-second start delay, emergency-stop priority and start-cycle counting.

1. Start delay timing when both DO_LOW and AERATION_E_STOP_NC are TRUE. Allow BLOWER_RUN to become TRUE only after these conditions have remained satisfied for 5 seconds.
2. If DO_LOW becomes FALSE or an emergency stop occurs, immediately disable BLOWER_RUN and reset the start timer, whether timing is incomplete or complete. A new demand must wait the full 5 seconds again.
3. Increment AERATION_BLOWER_CYCLE_COUNT once on each FALSE-to-TRUE transition of BLOWER_RUN. Do not count repeatedly during continuous operation.
4. At power-up, initialize the blower to OFF and the start count to zero. The start delay gates a BOOL enable; no continuous speed ramp is required.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`DO_LOW`	`BOOL`	Dissolved oxygen is below its threshold and aeration is needed.
Input	`AERATION_E_STOP_NC`	`BOOL`	TRUE means the emergency stop is inactive; FALSE means it is active.

Direction	Variable/Field	Data Type	Description
Output	`BLOWER_RUN`	`BOOL`	Blower motor-contactor run command.
Output	`AERATION_BLOWER_CYCLE_COUNT`	`UINT`	Count of actual blower starts.

---

07 · Multi-Pump Switching Control System

- **Difficulty:** Medium (medium)

Requirements

In an industrial water supply system, there are 5 pumps, each of which can be controlled independently. The system needs to support both manual and automatic operating modes.
Based on the value of the mode variable, switch to the corresponding operating mode.
In manual mode, control the start and stop of the selected pump based on the value of pumpGroup[1..5].selection.
In automatic mode, compare the priority of each pump, and automatically select the 3 pumps with higher priority for water supply according to the method of three in use and two on standby.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`mode`	`BOOL`	FALSE: Manual mode TRUE: Automatic mode
Input	`start`	`BOOL`	Start button, used to start the manually selected or automatically selected pump
Input	`stop`	`BOOL`	Stop button, used to stop all pumps
Input/Output	`pumpGroup`	`ARRAY[1..5] OF ST_Pump`	Five pump records passed by reference. Read priority and selection; write only RunComd.

Data type `ST_Pump`

Field	Data Type	Description
`priority`	`INT`	Priority; larger values have higher priority. Read-only to the function block.
`selection`	`BOOL`	Manual-selection state. Read-only to the function block.
`RunComd`	`BOOL`	Run command written by the function block; TRUE runs and FALSE stops.

---

08 · Analog Batch Processing

- **Difficulty:** Medium (medium)

Requirements

Write a PLC function block that processes analog inputs in batches and converts them into engineering values.

1. analogValues supplies the raw-value array. channels supplies per-channel settings and receives conversion results. count specifies how many channels to process from the beginning of the arrays. The caller must provide count from zero through the smaller of the two array lengths. Match channels by ordinal position starting at each array's own lower bound. Read analogValues without modifying it.
2. Each channel is enabled by its enable field and converted according to measuringMode. Read configuration fields without modifying them; write only retVal and outputValue.
3. For measuringMode 0, perform non-temperature linear scaling: when bipolar is FALSE, map raw values 0..27648 to loLIM..hiLIM; when TRUE, map -27648..27648 to loLIM..hiLIM.
4. For measuringMode 1, perform standard-temperature conversion: outputValue is one tenth of the raw analog value.
5. For measuringMode 2, perform climatic-temperature conversion: outputValue is one hundredth of the raw analog value.
6. If measuringMode is not 0, 1 or 2, set that channel's retVal to 16#8001 and skip its conversion.
7. If hiLIM is less than or equal to loLIM, set that channel's retVal to 16#8002 and skip its conversion.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`count`	`DINT`	Number of channels to be converted
Input/Output	`analogValues`	`ARRAY[*] OF INT`	Variable-length analog array passed by reference to preserve its bounds. Read-only to the function block; no element may be modified.

Direction	Variable/Field	Data Type	Description
Input/Output	`channels`	`ARRAY[*] OF ST_AnalogChannel`	Channel records passed by reference. Configuration fields are read-only; write only retVal and outputValue.
Input field	`channels[].enable`	`BOOL`	Channel conversion enable. Read-only to the function block.
Input field	`channels[].hiLIM`	`REAL`	Upper engineering-unit limit. Read-only to the function block.
Input field	`channels[].loLIM`	`REAL`	Lower engineering-unit limit. Read-only to the function block.
Input field	`channels[].bipolar`	`BOOL`	FALSE selects unipolar and TRUE selects bipolar operation. Read-only to the function block.
Input field	`channels[].measuringMode`	`INT`	0 non-temperature, 1 standard temperature, 2 climatic temperature. Read-only to the function block.
Output field	`channels[].retVal`	`WORD`	Per-channel status code written by the function block.
Output field	`channels[].outputValue`	`REAL`	Converted value written by the function block.

Rows named `channels[].fieldName` describe the element fields of ST_AnalogChannel, rather than additional function-block parameters.

09 · Calculate Moving Average

- **Difficulty:** Medium (medium)

Requirements

Write a function block FB that calculates and updates a moving arithmetic mean. The moving arithmetic mean is a method of averaging over consecutive data points, where each new data point replaces the oldest data point, and then the average is recalculated.

1. When cyclicExecution is TRUE, a new data value is automatically read and the moving average average is updated every scan cycle.
2. Provide an external trigger signal trigger, which reads a new data value and updates the moving average upon each rising edge.
3. Provide a reset function reset, which resets the moving average calculation when the reset signal is TRUE. If the window size is less than 1 or greater than 100, output an error status and status code.

The status parameter indicates the execution status of the program:

- 16#0000: Execution successful
- 16#8200: Window length setting error, please set a value between 1 and 100.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`cyclicExecution`	`BOOL`	When TRUE, periodic reading is performed, and trigger has no effect
Input	`trigger`	`BOOL`	External trigger signal, reads value upon each rising edge
Input	`value`	`LREAL`	Newly read data value
Input	`windowSize`	`INT`	The window size for the moving average, which must be between 1 and 100
Input	`reset`	`BOOL`	Reset signal, resets the moving average calculation when TRUE
Output	`average`	`LREAL`	The moving average
Output	`windowSizeReached`	`BOOL`	FALSE: Maximum window width not yet reached TRUE: Maximum window width has been reached
Output	`error`	`BOOL`	FALSE: No error occurred TRUE: An error occurred during the execution of FB
Output	`status`	`WORD`	Status code

10 · Temperature Logic Control

- **Difficulty:** Medium (medium)

Requirements

Write a function block FB to implement a temperature control logic. The program should include the following functions:

1. Read the value of the temperature sensor through an analog input module.
2. Compare the read temperature value TemperatureSensorInput with the set temperature value SetTemperature.
3. If the actual temperature is lower than the set temperature, start the heater HeaterOutput.
4. If the actual temperature is higher than the set temperature, turn off the heater HeaterOutput.
5. If the heater works continuously for more than a certain period (for convenience, using 5s), it should trigger an overheating protection, automatically turn off the heater, and must wait for a certain period (for convenience, using 3s) before the overheating protection can be lifted.
6. Provide status indication, displaying the current temperature, set temperature, heater status, and whether overheating protection is triggered.
7. Provide temperature sensor fault judgment, if the read temperature value TemperatureSensorInput is less than 0 or greater than 100, it is considered a sensor fault, and the heater should be turned off.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`TemperatureSensorInput`	`REAL`	The input value of the temperature sensor, i.e., the actual temperature
Input	`SetTemperature`	`REAL`	The set temperature value
Output	`CurrentTemperature`	`REAL`	Display of the current temperature
Output	`SetTempDisplay`	`REAL`	Display of the set temperature
Output	`HeaterStatus`	`BOOL`	Display of the heater status
Output	`OverheatProtection`	`BOOL`	Display of the overheating protection status
Output	`HeaterOutput`	`BOOL`	The control output of the heater

---

11 · Automated Production Line Control

- **Difficulty:** Medium (medium)

Requirements

A simple automated production line consists of three workstations: Workstation A, Workstation B, and Workstation C. The product needs to go through these three stations in sequence to be considered fully completed. There is a motor in the production line that is responsible for moving the product from one workstation to the next. The sequence of the line is from Workstation A to Workstation B, and then to Workstation C. Each workstation has a sensor (used to detect if the product has arrived) and a button (used to trigger the signal that the station's processing is complete).
Control requirements:

1. Mode Switching: The mode switch (Mode) is used to switch between manual and automatic modes of the production line.
2. Manual Mode: In manual mode, the forward rotation button (ForwardButton) controls the forward rotation of the motor, and the reverse rotation button (ReverseButton) controls the reverse rotation of the motor. Forward and reverse rotation operations need to be interlocked and cannot be performed simultaneously.
3. Automatic Mode:
    - When the product arrives at Workstation A, SensorA detects the product, and after the operator triggers the processing complete signal via ButtonA, the motor starts forward rotation to move the product to Workstation B.
    - When the product arrives at Workstation B, SensorB detects the product, the motor stops, and after the operator triggers the processing complete signal via ButtonB, the motor starts forward rotation to move the product to Workstation C.
    - When the product arrives at Workstation C, SensorC detects the product, the motor stops, and after the operator triggers the processing complete signal via ButtonC, the completion indicator light (CompletionLight) turns on, indicating that the product processing is complete. When the product is taken away, i.e., SensorC no longer detects the product, the completion indicator light (CompletionLight) turns off.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`Mode`	`BOOL`	Used to switch the production line between manual and automatic modes, 0=manual mode, 1=automatic mode
Input	`ForwardButton`	`BOOL`	Forward rotation button
Input	`ReverseButton`	`BOOL`	Reverse rotation button
Input	`SensorA`	`BOOL`	Sensor A
Input	`SensorB`	`BOOL`	Sensor B
Input	`SensorC`	`BOOL`	Sensor C
Input	`ButtonA`	`BOOL`	Button A
Input	`ButtonB`	`BOOL`	Button B

Direction	Variable/Field	Data Type	Description
Input	`ButtonC`	`BOOL`	Button C
Output	`MotorForward`	`BOOL`	Motor forward rotation output
Output	`MotorReverse`	`BOOL`	Motor reverse rotation output
Output	`CompletionLight`	`BOOL`	Indicator light for product processing completion

---

12 · Slab Thickness Alarm and Fault History

- **Difficulty:** Medium (medium)

Requirements

Monitor slab-thickness faults on a stone-cutting line, including minimum alarm holding, fault counting, normal-cutting runtime accumulation and a history of the five most recent faults.

1. When THICKNESS_SIGNAL_VALID is TRUE, THICKNESS_FAULT indicates that thickness deviation exceeds tolerance. Activate THICKNESS_ALARM immediately when the fault becomes TRUE, and hold each triggered alarm for at least 500 ms. Keep the alarm active while the fault persists.
2. On each FALSE-to-TRUE transition of a valid fault signal, increment Slab_Fault_Counter and store the current FaultTimestamp in element 0 of Slab_Fault_History. Shift older entries toward higher indices, retaining only the five most recent entries.
3. Accumulate Slab_Runtime_Accumulator using CycleTime only while CUTTING_RUN is TRUE, the thickness signal is valid and no thickness fault is present. Otherwise preserve the accumulated value.
4. At power-up or whenever THICKNESS_SIGNAL_VALID is FALSE, switch off the alarm and clear its holding state. Do not add fault records or accumulate normal runtime while the signal is invalid.
5. At power-up, initialize the fault count, accumulated runtime and all five history entries to zero. Statistics and history are for observation and maintenance only; they must not alter the cutting equipment's operating permissive, and external callers must not modify history entries.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`THICKNESS_FAULT`	`BOOL`	Thickness deviation exceeds the permitted tolerance.
Input	`THICKNESS_SIGNAL_VALID`	`BOOL`	Thickness-signal validity; FALSE means the signal is lost or unavailable.

Direction	Variable/Field	Data Type	Description
Input	`CUTTING_RUN`	`BOOL`	Actual cutting-equipment running status.
Input	`FaultTimestamp`	`DINT`	Current timestamp in seconds, used as the fault-record identifier.
Input	`CycleTime`	`TIME`	Actual elapsed interval represented by this invocation; must be positive.
Output	`THICKNESS_ALARM`	`BOOL`	Thickness alarm with a minimum 500 ms hold duration.
Output	`Slab_Fault_Counter`	`UINT`	Number of valid thickness-fault occurrences.
Output	`Slab_Runtime_Accumulator`	`LTIME`	Accumulated normal-cutting runtime.
Output	`Slab_Fault_History`	`ARRAY[0..4] OF DINT`	Five most recent fault timestamps; index 0 is newest; read-only to callers.

---

13 · Office HVAC Multi-mode Energy-Saving Control

- **Difficulty:** Medium (medium)

Requirements

Control an office-building supply fan and generate a temperature-setpoint adjustment from occupancy, operating mode and the energy-saving selection.

1. Valid HVAC_MODE_SELECTOR values are 0 (Auto), 1 (Comfort), 2 (Economy) and 3 (Night).
2. Start a 30-second delay only when occupancy is detected and Night mode is not selected. SUPPLY_FAN_RUN becomes TRUE only after these conditions remain satisfied for 30 seconds. Immediately stop the fan and reset timing if occupancy is lost or Night mode is selected. Wait the full 30 seconds again when the conditions recover.
3. Auto mode: use a 0°C adjustment when occupied. When unoccupied, use +3°C in energy-saving mode or +2°C in standard mode.
4. Comfort mode: use 0°C when occupied or +1°C when unoccupied. Economy mode: use 0°C when occupied or +4°C when unoccupied.
5. Night mode: use +4°C regardless of occupancy. The energy-saving selection affects only Auto mode.

6. Keep TEMP_SETPOINT_ADJUST within ±4°C. Initialize outputs to OFF and zero at power-up; update them from current inputs when normal calculation begins.
7. The start delay controls only the BOOL fan enable; no continuous speed ramp is required.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`OCCUPANCY_DETECTED`	`BOOL`	Occupancy-detection result derived from the CO_2 sensor.
Input	`ENERGY_SAVING_MODE`	`BOOL`	TRUE selects energy saving; FALSE selects standard operation; affects Auto mode only.
Input	`HVAC_MODE_SELECTOR`	`INT`	0 Auto, 1 Comfort, 2 Economy, 3 Night; callers supply a valid value.
Output	`SUPPLY_FAN_RUN`	`BOOL`	Supply-fan run enable.
Output	`TEMP_SETPOINT_ADJUST`	`INT`	Temperature-setpoint adjustment in °C.

---

14 · IPv4 Parsing with Format and Range Checks

- **Difficulty:** Medium (medium)

Requirements

Design a PLC function block in Structured Text (ST) that parses a dotted-decimal IPv4 string into a 32-bit unsigned integer.

1. ipText must contain four nonempty decimal-digit segments separated by periods. Each segment must have a value from 0 through 255.
2. Assemble the address in network byte order: the first segment occupies the most significant 8 bits and the fourth occupies the least significant 8 bits. Write the result to ipValue.
3. Set ipValue to 0 for malformed input, a nondigit segment, an incorrect segment count or any out-of-range segment. The valid address 0.0.0.0 also produces 0.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`ipText`	`STRING(254)`	IPv4 text to parse, with a capacity of 254 characters. A valid dotted-decimal address requires at most 15 characters.
Output	`ipValue`	`UDINT`	32-bit address in network byte order; zero for invalid input.

---

15 · Material Proportioning Process Control

- **Difficulty:** Hard (hard)

Requirements

Design a PLC program to control a material proportioning process that requires mixing three types of materials according to specific requirements and then delivering them to the next process through valve D. The system needs to support both manual and automatic operation modes and be able to safely stop all equipment when an emergency stop signal is triggered.
Control requirements:
Emergency Stop Logic: The estop(=FALSE) signal is used for emergency stopping of all equipment.
Manual Mode: opeMode=false, manually control the start/stop of valves A, B, C, D, and the mixing motor.
Automatic Mode: opeMode=true, equipment operates automatically according to the process flow.
Implement the emergency stop logic to ensure that all equipment stops immediately when the emergency stop signal is triggered.
In manual mode, manually control the start/stop of valves and the mixing motor.
For example, pressing the valve A start button valveAStart (triggering a rising edge) causes valve A to run (valveARun=TRUE); pressing the valve A start button valveAStart again (triggering a rising edge) stops valve A (valveARun=FALSE).
In automatic mode, implement the following control logic based on the process flow:
Press the autoStart button to start the automatic process, first opening valve A.
When the liquid level sensor levelSensor reaches 50cm, close valve A and open valve B.
When the liquid level sensor reaches 80cm, close valve B.
Based on processMode:
If processMode=false, start the mixing motor; after receiving the mixing complete signal, stop the mixing motor, open valve D, and the automatic process ends.
If processMode=true, open valve C; when the liquid level sensor reaches 100cm, close valve C, start the mixing motor; after receiving the mixing complete signal, stop the mixing motor, open valve D, and the automatic process ends.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`estop`	`BOOL`	Emergency Stop Signal FALSE: Emergency stop triggered TRUE: No emergency stop triggered
Input	`opeMode`	`BOOL`	Operation Mode FALSE: Manual Operation Mode TRUE: Automatic Operation Mode
Input	`valveAStart`	`BOOL`	Valve A Start Button
Input	`valveBStart`	`BOOL`	Valve B Start Button
Input	`valveCStart`	`BOOL`	Valve C Start Button
Input	`valveDStart`	`BOOL`	Valve D Start Button
Input	`mixMotorStart`	`BOOL`	Mixing Motor Start Button

Direction	Variable/Field	Data Type	Description
Input	`autoStart`	`BOOL`	Automatic Start Button
Input	`processMode`	`BOOL`	Process Mode
Input	`levelSensor`	`INT`	Liquid Level Sensor (unit cm)
Input	`mixMotorComplete`	`BOOL`	Mixing Complete Signal
Output	`valveARun`	`BOOL`	Valve A Run Command
Output	`valveBRun`	`BOOL`	Valve B Run Command
Output	`valveCRun`	`BOOL`	Valve C Run Command
Output	`valveDRun`	`BOOL`	Valve D Run Command
Output	`mixMotorRun`	`BOOL`	Mixing Motor Run Command

---

16 · Flexible Processing Control

- **Difficulty:** Hard (hard)

Requirements

An automated production line consists of 5 workstations: numbered 1# to 5# from left to right. The 1# station is used for raw material loading, stations 2# to 4# are used for product processing, and the 5# station is used for finished product output and storage.
There is a motor in the production line, responsible for moving the product from one workstation to the next. Each workstation has a sensor (used to detect if the product has arrived).
Each product has a different sequence and number of processing steps, specified by the process sequence 'processSequence'. The processSequence is an array type, storing the sequence numbers of the workstations, and the system automatically sends the raw materials to the corresponding workstation for processing based on the station number.
Control requirements:
At the start of production, when the sensor at the 1# station (sensor1) detects the raw material, the start button at the 1# station (station1Start) is pressed. The system queries the first element of the process sequence 'processSequence', starts the conveyor to move the raw material to the corresponding workstation, and stops after delivery. Upon receiving the processing completion signal from the corresponding workstation (station2Complete to station4Complete), it continues to query the next element of the process sequence 'processSequence', determines the direction of the conveyor, and starts the conveyor (conveyorLeft, conveyorRight). This continues until the product is delivered to the 5# workstation.

processLength specifies the active sequence length, from 1 through 8, starting at processSequence[1]. The final active entry is output station 5; other active entries are processing stations 2 through 4. The caller supplies a sequence satisfying this contract.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`processLength`	`USINT`	Active process-sequence length, 1..8; the final active entry must be station 5.
Input	`sensor1`	`BOOL`	Sensor at 1# station
Input	`sensor2`	`BOOL`	Sensor at 2# station
Input	`sensor3`	`BOOL`	Sensor at 3# station
Input	`sensor4`	`BOOL`	Sensor at 4# station
Input	`sensor5`	`BOOL`	Sensor at 5# station
Input	`station1Start`	`BOOL`	Start button at 1# station
Input	`station2Complete`	`BOOL`	Processing completion signal at 2# station
Input	`station3Complete`	`BOOL`	Processing completion signal at 3# station
Input	`station4Complete`	`BOOL`	Processing completion signal at 4# station
Input	`processSequence`	`ARRAY[1..8] OF USINT`	Processing sequence
Output	`conveyorLeft`	`BOOL`	Conveyor turns left
Output	`conveyorRight`	`BOOL`	Conveyor turns right

---

17 · Recipe Management System

- **Difficulty:** Hard (hard)

Requirements

A factory's production line needs to produce a variety of products based on different recipes. The control center of the production line requires a recipe management system that allows operators to add, delete, modify, and query recipe data.
Control requirements:
Recipe data should include recipe number, raw material type, raw material ratio, and production temperature.
The system should allow input of new recipe data and store it in the recipe database.
The system should allow deletion or modification of existing recipe data by recipe number.
The system should allow querying recipe details by recipe number and output the recipe.
Notes:

1. Recipes are constructed using the ST_Recipe data type, which includes recipe number (recipeID), raw material type (ingredientType), raw material ratio (ingredientRatio), and production temperature (productionTemperature). The recipe database can store up to 10 recipes, constructed with ARRAY[1..10] OF ST_Recipe, where recipe numbers must not be duplicated, and a recipe number of 0 indicates an empty slot.
2. When a query recipe request signal (queryRecipe) is triggered, the system first compares the recipe number (recipeID) from the input recipe (recipeIn) with all recipes in the recipe database recipe. If the number does not exist, an error status (error=true) and error code (status=16#8001) are output. If the number exists, the recipe is output to the recipe query result (recipeQueryResult).
3. When an add recipe request signal (addRecipe) is triggered, the system first compares the recipe number (recipeID) from the input recipe (recipeIn) with all recipes in the recipe database (recipe). If a duplicate number exists, an error status (error=true) and error code (status=16#8002) are output. If there is no empty slot in the recipe database (recipe), an error status (error=true) and error code (status=16#8003) are output. If there are no errors, the input recipe (recipeIn) is stored in the recipe database (recipe), prioritizing the empty slot with the smallest array index.
4. When a delete recipe request signal (deleteRecipe) is triggered, the system first compares the recipe number (recipeID) from the input recipe (recipeIn) with all recipes in the recipe database (recipe). If the number does not exist, an error status (error=true) and error code (status=16#8004) are output. If the number exists, the recipe is deleted, i.e., the recipe number (recipeID) in the recipe database (recipe) is set to 0.
5. When a modify recipe request signal (modifyRecipe) is triggered, the system first compares the recipe number (recipeID) from the input recipe (recipeIn) with all recipes in the recipe database (recipe). If the number does not exist, an error status (error=true) and error code (status=16#8005) are output. If the number exists, the recipe is replaced with the input recipe (recipeIn).

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`addRecipe`	`BOOL`	Add Recipe Request Signal
Input	`deleteRecipe`	`BOOL`	Delete Recipe Request Signal
Input	`modifyRecipe`	`BOOL`	Modify Recipe Request Signal
Input	`queryRecipe`	`BOOL`	Query Recipe Request Signal
Input	`recipeIn`	`ST_Recipe`	Input Recipe
Output	`recipeAdded`	`BOOL`	Recipe Added Success Indicator
Output	`recipeDeleted`	`BOOL`	Recipe Deleted Success Indicator

Direction	Variable/Field	Data Type	Description
Output	`recipeModified`	`BOOL`	Recipe Modified Success Indicator
Output	`recipeQueryResult`	`ST_Recipe`	Recipe Query Result Output
Output	`error`	`BOOL`	Error Status Indicator FALSE: No Error Occurred TRUE: Execution Error
Output	`status`	`WORD`	Status Code
Input/Output	`recipe`	`ARRAY[1..10] OF ST_Recipe`	Recipe Database

Data type `ST_Recipe`

Field	Data Type	Description
`recipeID`	`INT`	Recipe identifier; zero denotes an unused database slot.
`ingredientType`	`INT`	Ingredient-type identifier.
`ingredientRatio`	`REAL`	Ingredient proportion.
`productionTemperature`	`REAL`	Production temperature.

---

18 · FIFO First-In-First-Out Queue

- **Difficulty:** Hard (hard)

Requirements

Write a function block FB implementing a First-In-First-Out (FIFO) circular queue. Derive the queue capacity and element type from the caller-supplied buffer.

1. Each rising edge of enqueue requests one enqueue operation. If the queue is not full, append the current item value to its tail.
2. Each rising edge of dequeue requests one dequeue operation. If the queue is not empty, remove its front element and write that value back to item. The caller must not request enqueue and dequeue in the same invocation.
3. When reset is TRUE, empty the queue and set elementCount to zero. Reset takes priority over enqueue and dequeue; stored buffer values need not be overwritten.
4. Output the current number of valid elements through elementCount. Zero means the queue is empty; a count equal to the actual buffer array length means it is full.
5. Reject a dequeue request on an empty queue or an enqueue request on a full queue. Leave the queue and item unchanged and report the reason through status.

6. The queue is empty on its first invocation. Define head and tail positions and other internal state within the implementation; do not expose them as interface parameters.

Status codes:

- 16#0000: No error in this invocation, including idle, reset and successful operations.
- 16#8001: The dequeue request was rejected because the queue is empty.
- 16#8002: The enqueue request was rejected because the queue is full.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`enqueue`	`BOOL`	Enqueue request; one operation per rising edge.
Input	`dequeue`	`BOOL`	Dequeue request; one operation per rising edge.
Input	`reset`	`BOOL`	TRUE resets the queue to empty; takes priority over enqueue and dequeue.
Output	`elementCount`	`DINT`	Current number of valid elements, from zero through the queue capacity.
Output	`status`	`WORD`	Invocation status: 16#0000 no error, 16#8001 dequeue from an empty queue, 16#8002 enqueue into a full queue.
Input/Output	`item`	`VARIANT`	Mutable scalar reference: read its value when enqueuing; write the removed front value on successful dequeue. Its concrete type must match the array element type.
Input/Output	`buffer`	`VARIANT`	Reference to a nonempty one-dimensional array with a single concrete element type, used as queue storage. Capacity is the actual array length.

Associate buffer with a nonempty, homogeneous one-dimensional array and item with a writable scalar of the same element type. Keep the buffer association fixed for each function-block instance. Derive capacity from the actual array length; it must fit in a positive DINT.

---

19 · Mechanical Arm Control

- **Difficulty:** Hard (hard)

Requirements

Design a PLC program to control an automated mechanical arm that performs handling tasks on the production line. The mechanical arm has six basic actions: down, up, left, right, grip, and release. The operating modes of the mechanical arm include four types: manual, single step, single cycle, and continuous.
Control requirements:

1. Manual mode: Allows the operator to control each action of the mechanical arm individually through the control panel, provided it is not yet in position. For example, pressing the down button (DownButton=1), if not at the down position (DownLimitSwitch=0), then move down (MoveDown=1); if at the down position (DownLimitSwitch=1), stop moving down. Releasing the down button also stops the movement down.
2. Single step mode: The mechanical arm automatically executes one action according to the preset program, and the operator needs to confirm after each action is completed to execute the next action.
   The initial position of the mechanical arm is at the top left.
   When the start button is pressed once, the mechanical arm executes the down command MoveDown, and stops after reaching the down position.
   When the start button is pressed again, the mechanical arm executes the grip command ActivateGrip, and stops after gripping.
   When the start button is pressed again, the mechanical arm executes the up command MoveUp, and stops after reaching the up position.
   When the start button is pressed, the mechanical arm executes the right command MoveRight, and stops after reaching the right position.
   When the start button is pressed, the mechanical arm executes the down command MoveDown, and stops after reaching the down position.
   When the start button is pressed, the mechanical arm executes the release command ActivateRelease, and stops after releasing.
   When the start button is pressed again, the mechanical arm executes the up command MoveUp, and stops after reaching the up position.
   When the start button is pressed again, the mechanical arm executes the left command MoveLeft, and stops after reaching the left position.
   Then starts the cycle again from the beginning
3. Single cycle mode: The mechanical arm automatically completes a full handling cycle, including all preset actions, and stops at the end of the cycle waiting for the operator's instruction.
   The initial position of the mechanical arm is at the top left, when the start button is pressed: The mechanical arm first moves down, then grips after reaching the down position, moves up after gripping, moves right after reaching the up position, moves down after reaching the right position, releases after reaching the down position, moves up after releasing, moves left after reaching the up position, and stops after reaching the left position.
4. Continuous mode: The initial position of the mechanical arm is at the top left, when the start button is pressed, the mechanical arm continuously executes the handling cycle of the single cycle mode.
5. The emergency stop function can stop all actions of the mechanical arm at any time.

Use GripLimitSwitch and ReleaseLimitSwitch as the completion feedback for gripping and releasing respectively. The caller keeps the four mode-selection inputs mutually exclusive.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`ManualMode`	`BOOL`	Manual mode
Input	`SingleStepMode`	`BOOL`	Single step mode
Input	`SingleCycleMode`	`BOOL`	Single cycle mode
Input	`ContinuousMode`	`BOOL`	Continuous mode
Input	`EmergencyStop`	`BOOL`	Emergency stop signal FALSE: Emergency stop triggered TRUE: No emergency stop triggered
Input	`Start`	`BOOL`	Start operation
Input	`DownButton`	`BOOL`	Down button
Input	`UpButton`	`BOOL`	Up button
Input	`LeftButton`	`BOOL`	Left button
Input	`RightButton`	`BOOL`	Right button
Input	`GripButton`	`BOOL`	Grip button
Input	`ReleaseButton`	`BOOL`	Release button
Input	`DownLimitSwitch`	`BOOL`	Down position limit switch
Input	`UpLimitSwitch`	`BOOL`	Up position limit switch
Input	`LeftLimitSwitch`	`BOOL`	Left position limit switch
Input	`RightLimitSwitch`	`BOOL`	Right position limit switch
Input	`GripLimitSwitch`	`BOOL`	Grip position limit switch
Input	`ReleaseLimitSwitch`	`BOOL`	Release position limit switch
Output	`MoveDown`	`BOOL`	Down command
Output	`MoveUp`	`BOOL`	Up command
Output	`MoveLeft`	`BOOL`	Left command
Output	`MoveRight`	`BOOL`	Right command
Output	`ActivateGrip`	`BOOL`	Grip command
Output	`ActivateRelease`	`BOOL`	Release command

20 · Production Control

- **Difficulty:** Hard (hard)

Requirements

An automated production line consists of 5 workstations: material loading station, cleaning station, weighing station, firing station, and finished product station. The production line has a motor responsible for moving products from one workstation to the next, the sequence of the line is from the material loading station, cleaning station, weighing station, firing station, to the finished product station. Each workstation has a sensor (used to detect if the product has arrived). Control requirements:

1. Mode Switching: The mode switch (Mode) is used to switch between manual and automatic modes of the production line. 0 = manual mode, 1 = automatic mode.
2. Manual Mode: In manual mode, the forward rotation button (ForwardButton) controls the forward rotation of the motor, and the reverse rotation button (ReverseButton) controls the reverse rotation of the motor. Forward and reverse rotation operations need to be interlocked and cannot be performed simultaneously.
3. Automatic Mode:
    - When the material arrives at the material loading station, the SensorFeed detects the material, the motor starts forward rotation, and moves the material to the cleaning station.
    - When the material arrives at the cleaning station, the SensorWash detects the material, the motor stops, and upon receiving the signal washComplete after cleaning is finished, the motor starts forward rotation, moving the material to the weighing station.
    - When the material arrives at the weighing station, the SensorWeight detects the material, the motor stops, and upon receiving weightComplete after weighing is complete, the material is judged to be qualified based on the weight value. If the weight value is greater than or equal to 100 and less than or equal to 200, it is qualified, otherwise not.
      If the material is unqualified, the rejection command kick is initiated.
      If the material is qualified, the firing temperature is calculated based on the weight value. The calculation rule is that the weight value from 100 to 200 linearly corresponds to the temperature value from 500 to 550. The firing temperature firingTemp is output. At the same time, the motor starts forward rotation, moving the material to the firing station.
    - When the product arrives at the firing station, the SensorFiring detects the material, the motor stops, and upon receiving the signal firingComplete after firing is finished, the motor starts forward rotation, moving the product to the finished product station.
    - When the product arrives at the finished product station, the SensorOutput detects the product, the motor stops, and the completion light (CompletionLight) turns on, indicating that the product processing is complete. When the product is taken away, i.e., the SensorOutput does not detect the product, the completion light (CompletionLight) turns off.

Interface Definition

Direction	Variable/Field	Data Type	Description
Input	`Mode`	`BOOL`	Used to switch the production line between manual and automatic modes, 0 = manual mode, 1 = automatic mode
Input	`ForwardButton`	`BOOL`	Forward rotation button
Input	`ReverseButton`	`BOOL`	Reverse rotation button
Input	`SensorFeed`	`BOOL`	Sensor at the material loading station detecting the arrival of materials
Input	`SensorWash`	`BOOL`	Sensor at the cleaning station detecting the arrival of materials
Input	`SensorWeight`	`BOOL`	Sensor at the weighing station detecting the arrival of materials
Input	`SensorFiring`	`BOOL`	Sensor at the firing station detecting the arrival of materials
Input	`SensorOutput`	`BOOL`	Sensor at the finished product station detecting the arrival of products
Input	`washComplete`	`BOOL`	Signal indicating completion of cleaning
Input	`weightComplete`	`BOOL`	Signal indicating completion of weighing
Input	`weight`	`REAL`	Weight value
Input	`firingComplete`	`BOOL`	Signal indicating completion of firing
Output	`MotorForward`	`BOOL`	Output for motor forward rotation
Output	`MotorReverse`	`BOOL`	Output for motor reverse rotation
Output	`kick`	`BOOL`	Rejection output
Output	`CompletionLight`	`BOOL`	Indicator light for product processing completion
Output	`firingTemp`	`REAL`	Firing temperature